

08. 영상 처리 응용

분류를 넘어 "어디에 무엇이 있는가"로 간다. 객체 검출(YOLO), 세그멘테이션(U-Net), 그리고 추론 성능을 끌어올리는 기법들.

1. 세 가지 과제의 차이

과제	출력	예시
분류	라벨 1개	"이 사진은 고양이"
검출	박스 + 라벨 여러 개	"여기에 고양이, 저기에 개"
세그멘테이션	픽셀마다 라벨	"이 픽셀은 고양이, 저 픽셀은 배경"

출력 형태가 달라지면 라벨 구조, 손실 함수, 평가 지표가 전부 달라진다. 이 장의 핵심이다.

2. 앵커 박스와 좌표 인코딩

검출 모델은 박스 좌표를 직접 예측하지 않는다. 미리 정해진 **앵커**로부터의 **상대적 차이**를 예측한다.

왜 상대값인가

이미지 크기가 416이면 좌표가 0~416 범위다. 그 큰 값을 직접 맞추게 하면 학습이 불안정하다. 앵커 대비 비율로 바꾸면 0 근처의 작은 값이 되어 다루기 쉽다.

```
def encode_boxes(anchors, gt_boxes):
    tx = (gt_boxes[:, 0] - anchors[:, 0]) / anchors[:, 2]
    ty = (gt_boxes[:, 1] - anchors[:, 1]) / anchors[:, 3]
    tw = torch.log(gt_boxes[:, 2] / anchors[:, 2])
    th = torch.log(gt_boxes[:, 3] / anchors[:, 3])
    return torch.stack([tx, ty, tw, th], dim = 1)

def decode_boxes(anchors, offsets):
    x = offsets[:, 0] * anchors[:, 2] + anchors[:, 0]
    y = offsets[:, 1] * anchors[:, 3] + anchors[:, 1]
    w = torch.exp(offsets[:, 2]) * anchors[:, 2]
    h = torch.exp(offsets[:, 3]) * anchors[:, 3]
    return torch.stack([x, y, w, h], dim = 1)
```

항목	인코딩	디코딩	이유
중심 좌표	$(gt - anchor) / 크기$	$offset \times 크기 + anchor$	앵커 크기로 정규화
폭·높이	$\log(gt / anchor)$	$\exp(offset) \times anchor$	항상 양수 보장

log 와 **exp** 를 쓰는 이유 — 폭과 높이는 음수가 될 수 없다. 로그 공간에서 예측하면 **exp** 를 거치며 자동으로 양수가 되고, 배율 변화가 대칭이 된다(2배 확대와 절반 축소가 +log2, -log2 로 크기가 같다).

검증 방법

인코딩 → 디코딩을 거치면 원본으로 돌아와야 한다.

```
encoded = encode_boxes(anchors, gt_boxes)
decoded = decode_boxes(anchors, encoded)
error = torch.abs(decoded - gt_boxes).mean() # 0에 가까워야 정상
```

이 왕복 검증이 구현 검사의 표준 방법이다.

앵커 생성

```
def generate_anchors(feature_size, image_size, scales, ratios):
    shift_y = torch.arange(0, fh) * (ih / fh) + (ih / fh) / 2
    shift_x = torch.arange(0, fw) * (iw / fw) + (iw / fw) / 2
    ...
    for scale in scales:
        for ratio in ratios:
            area = scale * scale
            w = torch.sqrt(torch.tensor(area / ratio))
            h = w * ratio
```

특징맵의 각 칸마다 여러 크기·비율의 박스를 깎는다. 개수는 곱으로 늘어난다.

항목	값
특징맵	8 × 8 = 64칸
스케일	3개 (32, 64, 128)
비율	3개 (0.5, 1.0, 2.0)
총 앵커	576개

+ (ih / fh) / 2 는 격자 칸의 **중심**으로 옮기는 부분이다. 이걸 빼면 앵커가 칸의 좌상단에 몰린다.

3. YOLO 레이어

특징맵을 박스 예측으로 바꾸는 층이다.

```
prediction = x.view(batch_size, self.num_anchors,
                    self.num_classes + 5, grid_size, grid_size)
prediction = prediction.permute(0, 1, 3, 4, 2).contiguous()

tx = torch.sigmoid(prediction[..., 0])
ty = torch.sigmoid(prediction[..., 1])
tw = prediction[..., 2]
th = prediction[..., 3]
conf = torch.sigmoid(prediction[..., 4])
pred_cls = torch.sigmoid(prediction[..., 5:])
```

마지막 층의 구성

num_classes + 5 에서 5가 무엇인지가 핵심이다.

위치	내용	활성화
0	중심 x 오프셋	sigmoid (격자 안으로 제한)
1	중심 y 오프셋	sigmoid
2	폭 (로그)	없음
3	높이 (로그)	없음
4	객체 신뢰도	sigmoid
5 이상	클래스 점수 전부	sigmoid

tx, ty 에 sigmoid를 씌우는 건 중심이 자기 격자 칸을 벗어나지 못하게 하기 위해서다. tw, th 는 로그 공간이라 그대로 둔다.

격자 좌표 더하기

```
pred_x = (tx + self.grid_x) / grid_size
pred_w = torch.exp(tw) * self.anchor_w / grid_size
```

tx 는 격자 칸 안에서의 위치(0~1)이고, grid_x 가 몇 번째 칸인지를 더한다. 마지막에 grid_size 로 나눠 0~1 정규 좌표로 만든다.

출력 크기 계산

항목	계산	값
입력	416×416	
MaxPool 4회	$416 \div 2^4$	그리드 26
앵커		3
예측 박스 수	$3 \times 26 \times 26$	2,028
출력		(2, 2028, 15)

15은 4(좌표) + 1(신뢰도) + 10(클래스) 이다.

4. U-Net 세그멘테이션

입력과 출력 크기가 같아야 하므로, **줄였다가 다시 늘리는** 대칭 구조를 쓴다.

```
class Up(nn.Module):
    def forward(self, x1, x2):
        x1 = self.up(x1)
        diffY = x2.size()[2] - x1.size()[2]
        diffX = x2.size()[3] - x1.size()[3]
        x1 = F.pad(x1, [diffX // 2, diffX - diffX // 2,
                       diffY // 2, diffY - diffY // 2])
        x = torch.cat([x2, x1], dim = 1)
        return self.conv(x)
```

Skip Connection이 핵심

인코더를 거치면 128×128이 8×8까지 줄어든다. "도로가 있다"는 정보는 남지만 **"경계가 어디냐"**는 사라진다. 디코더에서 다시 키워도 경계가 뭉개진다.

인코더의 고해상도 특징을 그대로 뒤로 넘겨 합치는 게 U-Net의 전부다.

ResNet과 헷갈리지 말 것		
모델	결합 방식	코드
ResNet	덧셈	out += shortcut(x)
U-Net	채널 연결	torch.cat([x2, x1], dim = 1)

F.pad 부분은 홀수 크기 때문에 1픽셀씩 어긋나는 경우를 메운다. 리스트 순서는 [좌, 우, 상, 하] 다.

채널 흐름 (bilinear = True , factor = 2)

단계	출력 채널	스킵	Up 입력
inc	64 (x1)		
down1	128 (x2)		
down2	256 (x3)		
down3	512 (x4)		
down4	512 (x5)		
up1(x5, x4)	256	512	512 + 512 = 1024
up2(·, x3)	128	256	256 + 256 = 512
up3(·, x2)	64	128	128 + 128 = 256
up4(·, x1)	64	64	64 + 64 = 128
outc	n_classes		64 입력

마지막 OutConv 는 kernel_size = 1 이다. **1×1 합성곱이 픽셀당 분류기** 역할을 한다.

5. Dice Loss

```
class DiceLoss(nn.Module):
    def forward(self, logits, targets):
        probs = torch.sigmoid(logits).view(-1)
        targets = targets.view(-1)
        intersection = (probs * targets).sum()
        union = probs.sum() + targets.sum()
        dice = (2.0 * intersection + self.smooth) / (union + self.smooth)
        return 1.0 - dice
```

왜 BCE가 아니라 Dice인가 — 세그멘테이션은 대부분 배경이다. 전체가 배경이라고 찍어도 BCE 손실이 꽤 낮게 나온다. Dice는 **겹치는 영역의 비율**을 보기 때문에 전경을 못 잡으면 바로 점수가 떨어진다.

`smooth` 를 분자와 분모 양쪽에 더하는 건 빈 마스크일 때 0으로 나누는 걸 막기 위함이다.

실무에서는 `BCE + Dice` 를 섞어 쓰는 경우가 많다.

6. TTA (Test Time Augmentation)

추론할 때 입력을 여러 형태로 변형해 예측한 뒤 평균 내는 기법이다. 학습을 더 하지 않고도 안정성을 올린다.

```
augmentations = [lambda x: torch.flip(x, dims = [-1]),
                  lambda x: torch.flip(x, dims = [-2]),
                  lambda x: torch.rot90(x, 1, dims = [-2, -1])]

reverse_augmentations = [lambda x: torch.flip(x, dims = [-1]),
                          lambda x: torch.flip(x, dims = [-2]),
                          lambda x: torch.rot90(x, -1, dims = [-2, -1])]
```

순서는 **변형 → 예측 → 되돌리기 → 평균**이다. 되돌리지 않으면 좌표계가 어긋난 예측들이 섞인다.

변형	역연산
<code>flip(dims=[-1])</code>	자기 자신
<code>flip(dims=[-2])</code>	자기 자신
<code>rot90(x, 1)</code>	<code>rot90(x, -1)</code>
<code>rot90(x, 3)</code>	<code>rot90(x, -3)</code>

뒤집기는 두 번 하면 원위치라 자기 자신이 역연산이지만, 회전은 **부호를 바꿔야** 한다.

멀티스케일 TTA

```
for scale in [0.8, 1.0, 1.2]:
    new_size = [int(s * scale) for s in original_size]
    scaled = F.interpolate(image.unsqueeze(0), size = new_size, ...)
    pred = test_time_augmentation(model, scaled.squeeze(0), ...)
    pred_resized = F.interpolate(pred, size = original_size, ...)
```

크기를 바꿔가며 예측하고 원래 크기로 되돌려 평균 낸다. 작은 객체와 큰 객체를 모두 잡는 데 도움이 된다.

7. 모델 앙상블

```
class ModelEnsemble(nn.Module):
    def __init__(self, models):
        super().__init__()
        self.models = nn.ModuleList(models)

    def forward(self, x):
        predictions = [model(x) for model in self.models]
        return torch.mean(torch.stack(predictions), dim = 0)
```

`nn.ModuleList` 가 핵심이다. 파이썬 리스트에 모델을 담으면 파라미터가 등록되지 않아 `.to(device)` 나 `.parameters()` 가 하위 모델을 못 찾는다.

방식	결과를 언제 합치나
결과 앙상블	각자 추론 후 밖에서 평균
모델 앙상블	하나의 <code>nn.Module</code> 안에서

후자는 전체를 하나의 모델처럼 저장·배포할 수 있다.

8. 멀티태스크 학습

하나의 백본을 공유하고 헤드를 여러 개 붙인다.

```
class MultiTaskModel(nn.Module):
    def forward(self, x):
        shared_features = self.shared_encoder(x)
        cls_output = self.classifier(shared_features)
        seg_output = self.segmentation(shared_features)
        return cls_output, seg_output
```

	장점	단점
멀티태스크	특징 추출을 공유해 효율적, 과제 간 상호 보완	메모리 소모, 과제 간 간섭

한정된 리소스로 여러 결과를 내야 할 때 쓴다.

헤드 구조 대비

헤드	구성	출력
분류	<code>AdaptiveAvgPool</code> → <code>Flatten</code> → <code>Linear</code>	<code>(B, num_classes)</code>
세그멘테이션	<code>Conv</code> + <code>Upsample</code> × 3	<code>(B, num_seg_classes, H, W)</code>

분류는 공간 정보를 버리고, 세그멘테이션은 복원한다. `MaxPool` 3회(1/8)를 `Upsample` 3회(8배)로 되돌려 원래 크기가 나온다.

손실 가중치

```
total_loss = self.alpha * loss_cls + self.beta * loss_seg
```

문제는 `alpha` 와 `beta` 를 사람이 정해야 한다는 점이다. 두 손실의 스케일이 다르면 한쪽이 학습을 독점한다.

불확실성 기반 자동 가중치

```
class UncertaintyWeightedLoss(nn.Module):
    def __init__(self):
        super().__init__()
        self.log_sigma_cls = nn.Parameter(torch.tensor(0.0))
        self.log_sigma_seg = nn.Parameter(torch.tensor(0.0))

    def forward(self, ...):
        precision_cls = torch.exp(-self.log_sigma_cls)
        weighted_cls = precision_cls * loss_cls + self.log_sigma_cls
        ...
```

가중치를 **학습으로 찾는다**. `log_sigma` 가 클수록 그 과제는 불확실하다고 보고 비중을 낮춘다.

뒤에 붙는 `+ self.log_sigma` 가 벌점 항이다. 이게 없으면 모델이 `log_sigma` 를 무한대로 키워 손실을 0으로 만들어 버린다.

옵티마이저에 **손실 함수의 파라미터도 넣어야** 학습된다.

```
optimizer = optim.Adam(list(model.parameters()) + list(criterion.parameters()), lr = 1e-3)
```

빠면 `log_sigma` 가 0.0에 고정되어 단순 합과 같아진다.

검수에서 찾은 동작 오류

1. 박스 인코딩 괄호 위치

```
tx = (gt_boxes[:, 0] - anchors[:, 0] / anchors[:, 2])      # 잘못
tx = (gt_boxes[:, 0] - anchors[:, 0]) / anchors[:, 2]      # 정상
```

나눗셈이 뺄셈보다 먼저 계산되어, 정규화가 아니라 엉뚱한 값이 나온다. 왕복 검증으로 바로 드러난다.

	정상	현재 코드
tx	약 0.1	103
디코딩 결과	105	5250

문법 오류가 아니라 **수식 오류**라 에러 없이 통과한다. 그래서 인코딩·디코딩 왕복 검증이 필요하다.

2. YOLO 클래스 슬라이싱

```
pred_cls = torch.sigmoid(prediction[..., 5])      # 5번 하나만
pred_cls = torch.sigmoid(prediction[..., 5:])     # 5번부터 끝까지
```

표기	결과 shape
[..., 5]	(B, A, G, G)
[..., 5:]	(B, A, G, G, num_classes)

콜론이 빠지면 클래스 점수를 하나만 꺼낸다. 마지막 view(batch, -1, num_classes) 에서 원소 개수가 안 맞아 터진다. num_classes = 1 일 때만 우연히 통과한다.

3. 박스 스택 축

```
pred_boxes = torch.stack([pred_x, pred_y, pred_w, pred_h], dim = 1)      # 잘못
pred_boxes = torch.stack([pred_x, pred_y, pred_w, pred_h], dim = -1)     # 정상
```

표기	결과
dim = 1	(B, 4, A, G, G) — 좌표가 흩어짐
dim = -1	(B, A, G, G, 4) — 정상

원소 개수는 같아서 **에러 없이 통과하지만** x, y, w, h 가 뒤섞인 채로 나온다. 조용히 잘못되는 종류라 더 위험하다.

4. U-Net 마지막 디코더 채널

```
self.up4 = Up(128, 64 // factor, bilinear)      # 잘못 → 32 출력
self.up4 = Up(128, 64, bilinear)                # 정상 → 64 출력
```

```
RuntimeError: weight of size [1, 64, 1, 1], expected input[2, 32, 256, 256]
to have 64 channels, but got 32 channels instead
```

up1 ~ up3 에 // factor 가 붙는 건 맞다. 다음 Up 이 스킵 연결과 합쳐 채널을 다시 채우기 때문이다.

그런데 up4 는 **마지막 디코더**라 뒤에 Up 이 없고 바로 OutConv(64, ...) 로 간다. 여기서 반으로 줄이면 받을 쪽과 어긋난다.

5. TTA 회전 역변환 부호

되돌리기 목적이 변형 목록과 **완전히 동일**하면, 뒤집기는 우연히 맞고 회전은 틀린다. 예측 결과가 엉뚱한 방향으로 회전된 채 평균에 들어간다. 에러가 안 난다.

6. 데이터셋 채널 수

```
image = torch.randn(2, self.img_size, self.img_size)      # 2채널
```

모델 첫 층이 nn.Conv2d(3, 64, ...) 라 3채널(RGB)을 기대한다. 학습 시작하자마자 채널 불일치로 터진다.

7. 시각화에서 예측 대신 입력을 다시 그림

```
pred_show = pred[0].squeeze().cpu().numpy()
axes[2].imshow(img_show)                        # 잘못 — 입력 이미지
axes[2].imshow(pred_show, cmap = "gray")        # 정상
```

pred_show 를 만들어놓고 쓰지 않아, "Prediction" 칸에 입력 이미지가 다시 그려진다. 에러가 안 나고 그래프도 나오지만 예측 결과는 화면에 없다.

8. Dataset 상속 누락

```
class MultiTaskDataset:          # 현재
class MultiTaskDataset(Dataset): # 권장
```

`__len__` 과 `__getitem__` 이 있으면 `DataLoader` 가 대체로 받아주지만, 파이토치 공식 방식은 상속이다.

정리 포인트

- 검출은 앵커 대비 **상대 좌표**를 예측. 폭·높이는 `log` 공간에서
- 인코딩 → 디코딩 **왕복 검증**이 구현 검사의 표준
- YOLO 마지막 축은 `4 + 1 + num_classes`. 클래스는 `[..., 5:]`
- U-Net은 `torch.cat` (연결), ResNet은 `+` (덧셈)
- Skip Connection이 **경계선 정보**를 살린다
- 마지막 `Up` 만 `// factor` 없이 — `OutConv` 가 받을 채널과 맞춰야 함
- 세그멘테이션은 배경이 대부분이라 **Dice Loss**가 유효
- TTA는 변형 → 예측 → **되돌리기** → 평균. 회전 역변환은 부호 반대
- 앙상블에는 반드시 `nn.ModuleList`
- 멀티태스크 손실 가중치는 `UncertaintyWeightedLoss` 로 학습시킬 수 있음. 옵티마이저에 손실 파라미터 포함 필수